DAY 07 · BUG CATALOG

The Bug Catalog

The planted bugs you must recognize on sight.

2–3 hours / day07_bug_catalog / Bug recognition

INSIDE THIS BOOKLET

Boundaries Logic Objects Bytes

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day07_bug_catalog/` · the course runner `python run.py 7`

In this session

Why a catalog works 3

A. Boundaries and indices (the #1 category) 3

B. Comparisons, logic and arithmetic 4

C. The Python object model 5

D. Strings and bytes 6

E. Algorithm logic and state 7

How to *find* them fast 8

Background for the exercises 8

The toolbox: Python you'll meet in today's exercises 10

Exercises 12

End-of-day checklist 13

◎ Today's goal

Recognise the bugs that get planted in "broken implementation" exercises *on sight*. Planted bugs are usually **one-token or one-line changes** that are easy to miss when you read: `<` vs `<=`, `i+1` vs `i`, a missing `.lower()`, the wrong variable, a swapped argument. Train your eyes to see them.

How to use this catalog

Read the catalog, then do the bug hunts. When you finish, add your own entries to your `bug_journal.md`.

Why a catalog works

Put yourself in the shoes of the person who prepares the exam. They start from a correct program and want to break it in a way that is *subtle*: the code should still look reasonable, still run, and still give the right answer on most inputs, failing only on particular tests. If they rewrote whole functions, the bugs would be obvious. So instead they make tiny edits: flip a comparison, shift an index by one, use the neighbouring variable, drop a copy, swap two arguments. There are only so many tiny edits that produce a believable bug, so the same few dozen patterns come up again and again. This catalog collects them. Once you know them, reading broken code stops being "stare until something looks odd" and becomes a targeted search: on each line you ask, "is this one of the known patterns?". Many entries repeat traps from Days 1–6. That's deliberate. This is the one place where they're all gathered together, grouped by *where* in the code they live. Each entry has a code (A1, B2, ...), and the exercises ask you to write that code next to each fix you make. That habit is what turns the catalog into a reflex.

A. Boundaries and indices (the #1 category)

Where they live: every loop, every `range`, every slice and every index has a start and an end, and each end can be off by one: included when it should be excluded, or the reverse. These bugs are the most common of all because they're literally one character (`<` vs `<=`, `+ 1` or nothing), and the code still runs. **What they look like in test results:** an `IndexError` at the end of the input; a result missing its first or last element; a number that's off by exactly one; or failures only on the edge cases, i.e. an empty input, a single element, or a value exactly at a limit. **How to check:** list every boundary in the function, and compare each one with the spec's wording. "Up to and including", "at most", "strictly greater", "the last one"... **A1 — `range` stops one too early.** `range(a, b)` never includes `b`. If the spec says "from 1 to n", the code needs `range(1, n + 1)`.

```python
def sum_to(n):
    """Sum of 1..n inclusive."""
    return sum(range(1, n))        # BUG: misses n; sum_to(3) gives 3 instead of 6
```

A2 — `<` vs `<=`. Whether a comparison includes equality changes the result exactly at the boundary. Typical places are binary search loops (`while lo < hi` vs `while lo <= hi`, explained at the end of this document) and interval checks: do `[1, 3]` and `[3, 5]` overlap? With `start < end` they don't, and with `start <= end` they do. Only the spec can tell you which is right. **A3 — reading one past the end.** Code that compares each item with the next one must stop one position early, because the last item has no "next":

```python
for i in range(len(x)):
    if x[i] == x[i + 1]:           # BUG: IndexError when i is the last position
        ...
for i in range(len(x) - 1):        # correct
```

A4 — skipping the last pair. The opposite mistake: stopping *two* early. With pairs `(i, i+1)`, the last pair starts at `len(x) - 2`, so the loop must be `range(len(x) - 1)`. `range(len(x) - 2)` silently ignores the final pair. No crash, just a missing result. **A5 — forgetting the leftovers after a loop.** Many loops collect items and only *emit* them when something changes, for example "when the next character is different, output the run so far". The very last item is never followed by a change, so it has to be emitted after the loop:

```
def runs(s):
    """'aabccc' -> ['aa', 'b', 'ccc']"""
    out, current = [], ""
    for ch in s:
        if current and ch != current[-1]:
            out.append(current)
            current = ""
        current += ch
    return out                    # BUG: the last run is lost; needs  if current:
        out.append(current)
```

The same happens when merging two sorted lists, where one list's tail is left over when the other runs out, and in parsers that "flush" the current word at the end. If the output is missing its *last* item, look for this. **A6 — wrong step after a match.** When replacing a pair of neighbours `(a, b)` in a list, after a match you must jump over *both* elements. Moving forward by only 1 lets the second element start another match:

```
# replace each (1, 1) with 9, left to right, without overlapping
i = 0
while i < len(ids):
    if i + 1 < len(ids) and ids[i] == 1 and ids[i + 1] == 1:
        out.append(9)
        i += 2                    # bug version: i += 1, so [1, 1, 1] gives [9, 9] instead of
            [9, 1]
    else:
        out.append(ids[i])
        i += 1
```

You'll see exactly this code in the Day 9 tokenizer. **A7 — `x[-n:]` when `n` is 0.** `-0` is just `0`, so `x[-0:]` is the whole list instead of an empty one (Day 1). The fix is `x[len(x) - n:]`, or a separate check for `n == 0`. **A8 — counting from 0 vs from 1.** Python counts positions from 0, but humans count lines from 1. If an error message should say "line 1" for the first line, the loop needs `enumerate(lines, start=1)`. Without `start=1`, every reported line number is one too small. **A9 — slice ends are exclusive.** `s[i:j]` stops *before* `j`, and `s[i:i+n]` is exactly `n` characters. Code that writes `s[i:j+1]` or `s[i:i+n-1]` "to be safe" is usually wrong by one. **A10 — off by one in numbering new items.** Specs often say exactly what number the first new item gets. In the BPE tokenizer, ids 0–255 are taken by single bytes, so the first learned token is 256, the next 257, and so on: `256 + i` for the i-th one, counting from 0. Variants like `255 + i` or `len(vocab) + 1` are typical planted bugs.

B. Comparisons, logic and arithmetic

Where they live: in the conditions and calculations that decide *which* answer is chosen, or *what value* is computed. **What they look like:** no crash, just a plausible but wrong answer: the second-best item instead of the best, the wrong one of two tied items, a total that's slightly off. **How to check:** work out the right answer by hand for the smallest failing input, then examine every `max`, `min`, `sorted` key, comparison operator and arithmetic operator on the path that produced the wrong answer. **B1 — the wrong direction.** `max` instead of `min`, `>` instead of `<`, or a missing (or extra) `reverse=True`. The result is the least frequent item when the spec wants the most frequent. **B2 — ties broken the wrong way.** When two items score the

same, *something* decides which one wins. `max(d, key=d.get)` returns whichever tied key comes first in the dict. If the spec says "on ties, the smallest key", that's only correct by luck. The tie rule has to be written into the key, as on Day 3: for example `min(d, key=lambda k: (-d[k], k))` means "highest value, and among equal values the smallest key". **B3 — `reverse=True` with a tuple key.** `reverse=True` reverses *the whole comparison*, so every part of the tuple gets reversed. For "count descending, name ascending", the key must be `(-count, name)` with no `reverse`. Writing `(count, name)` with `reverse=True` sorts tied names Z to A. **B4 — `and`/`or` mixed without parentheses.** Python evaluates `and` before `or`, so `a or b and c` means `a or (b and c)`. If the intent was `(a or b) and c`, the condition is wrong for some inputs. **B5 — the wrong kind of division.** `/` gives a float and `//` rounds down. `int(x)` rounds toward zero, which is different from `//` for negative numbers (`-7 // 2` is `-4`, but `int(-7 / 2)` is `-3`). See Day 1. **B6 — `round()` rounds halves to even.** `round(0.5)` is `0` and `round(2.5)` is `2` (Day 1). If the spec wants classic "round half up", the code needs something else, for example `int(x + 0.5)` for positive numbers. **B7 — comparing floats with `==`.** `0.1 + 0.2 == 0.3` is `False` (Day 1). Correct code compares with a small tolerance, or avoids floats altogether by counting in whole units (cents instead of euros). **B8 — `is` where `==` was meant.** `is` asks "the very same object?", and `==` asks "equal values?". `[1] == [1]` is `True`, but `[1] is [1]` is `False`, because they're two separate lists. Even for numbers, `x is 1000` can be `False` while `x == 1000` is `True`. The only place `is` belongs in normal code is `x is None`. **B9 — truthiness hiding a real zero.** `d.get(k) or default` returns the default when the stored value is `0` or `""`, even though the key exists, because `0 or default` evaluates to `default`. Likewise `if not count:` treats a legitimate count of 0 as "missing". See the stock example on Day 1, section 3. **B10 — the wrong variable.** A neighbouring variable is used by mistake: `i` instead of `j`, `a` twice instead of `a` and `b`. This is the hardest bug to *see*, because your eyes autocorrect to what they expect:

```
merged = vocab[a] + vocab[a]        # BUG: should be vocab[a] + vocab[b]
```

When a line looks fine but a test says otherwise, read it character by character, or say it out loud. **B11 — arguments in the wrong order.** `merge(ids, idx, pair)` instead of `merge(ids, pair, idx)`, or `isinstance(cls, obj)` instead of `isinstance(obj, cls)`. Check calls against the function's definition. **B12 — an inverted condition.** The condition says the opposite of what's meant:

```
if x in seen:                       # BUG: should be  not in
    seen.add(x)
    result.append(x)
```

Also `== 0x80` where `!= 0x80` was meant. If a branch seems to handle exactly the case it shouldn't, check for a missing or extra `not`. **B13 — the wrong bit operator or mask.** In bit-manipulation code (Day 5): `|` instead of `&`, `<<` instead of `>>`, the mask `0x1F` instead of `0x3F`, a shift by 8 instead of 6. These only break characters of a certain length, which is why the failing tests' inputs are your best clue.

C. The Python object model

Where they live: in how Python handles names, objects and references (Days 1 and 4). **What they look like:** a test that passes on its own but fails when run with the others; data that seems to change "by itself"; `None` where a value was expected; `AttributeError`; `TypeError: unhashable type`. **How to check:** whenever state behaves strangely, ask "who else holds a reference to this object?". **C1 — mutable default argument.** `def f(x, acc=[])` creates the list *once*, when the function is defined, so every call that doesn't pass `acc` shares it (Day 4, section 2). Use `acc=None` and create the list inside the function. **C2 — mutable class attribute.** A list or dict written directly in the class body (`class C: items = []`) is shared by every instance (Day 4, section 2). It must be created in `__init__` as `self.items = []`. **C3 — aliasing instead of copying.** `result = original` doesn't copy. It's a second name for the same object, so changing `result` changes `original`. The same goes for `self.items = items` when the spec says not to keep a reference to the caller's list. Copy with `list(x)`, `dict(x)` or `x.copy()`. **C4 — a shallow copy of nested data.** `dict(d)` and `list(l)` copy only the outer level. The inner lists and dicts are still shared,

and `[[0] * n] * m` repeats one row `m` times. Nested data that will be modified needs `copy.deepcopy`, or a comprehension that builds fresh inner objects. **C5 — using the return value of a method that returns `None`.** `x = lst.sort()`, `x = lst.append(1)` and `x = d.update(...)` all set `x` to `None`. Those methods change the object in place and return nothing. **C6 — calling a string method and throwing the result away.** `s.strip()`, `s.lower()` or `s.replace(...)` on a line of its own does nothing, because strings can't be changed and the new string is discarded (Day 2). It must be `s = s.strip()`. **C7 — a method mentioned but not called.** `line.strip` without parentheses is the method itself, not its result:

```
if line.strip:            # BUG: always True, because a method object is truthy
if line.strip():          # correct: True if the line has non-space content
```

C8 — `__eq__` without `__hash__`. Defining `__eq__` makes objects unhashable, so they can't go into sets or be dict keys, unless the class also defines `__hash__` (Day 4, section 5). **C9 — missing `super().__init__()`.** A subclass that defines its own `__init__` but doesn't call the base class's `__init__` never gets the base class's attributes, which leads to an `AttributeError` later (Day 4, section 6). **C10 — a property setter that assigns to itself.** Inside the setter for `x`, writing `self.x = value` calls the setter again, forever, until `RecursionError`. It must store into `self._x` (Day 4, section 3). **C11 — a generator used twice.** A generator is used up after one pass. Looping over it a second time gives nothing. The same is true of the objects returned by `zip()`, `map()` and `filter()` (Day 4, section 8). **C12 — changing a collection while looping over it.** Removing items from a list inside `for x in lst:` skips elements (Day 1). Adding or deleting dict keys inside `for k in d:` raises `RuntimeError` (Day 3). Loop over a copy, or build a new collection. **C13 — `return` or `yield` at the wrong indentation.** A `return` indented into the loop body returns after the first pass (Day 1, section 5). **C14 — using a loop variable after the loop.** After `for i in range(n):`, `i` keeps its last value, `n - 1`. But if the loop never ran (`n == 0`), `i` was never set at all:

```
for i in range(len(items)):
    ...
return i + 1              # works for non-empty lists, NameError/UnboundLocalError for an
    empty one
```

C15 — hiding a built-in function. Naming a variable `list`, `sum`, `id`, `max` or `bytes` hides the built-in function with that name, for the rest of the function:

```
sum = 0
for x in values:
    sum += x
average = sum / len(values)    # fine
total = sum(values)            # TypeError: 'int' object is not callable
```

C16 — catching too much. `except Exception: return None` swallows every error, including the ones the spec says must be raised. The test's `assertRaises` then fails, because nothing was raised (Day 4, section 7). **C17 — the wrong exception type.** Raising `KeyError` where the spec and tests want `ValueError`, or the other way round. Always check the test to see which type it expects.

D. Strings and bytes

Where they live: text processing, where the exact wording of the spec matters most: bytes or characters, which whitespace, which case, which format. **What they look like:** code that works on English text but fails on é; unexpected empty strings in a list; output that looks identical to the expected value but doesn't compare equal (compare their `repr`s, Day 2); the text `b'...'` appearing inside a string. **D1 — `split(" ")` instead of `split()`.** With two spaces in a row, tabs, or a trailing newline, `split(" ")` produces empty strings and keeps the newline attached to the last word (Day 2, section 2). **D2 — `strip("abc")` used to remove a word.** The argument is a *set of characters*, not a prefix. To remove an exact prefix once,

use `removeprefix` (Day 2). **D3 — case handled on only one side.** `if word.lower() == target:` is wrong when `target` might contain capitals. Both sides need `.lower()`. **D4 — counting characters where bytes were meant.** A length prefix in a binary format must count **bytes**: `len(s.encode("utf-8"))`, not `len(s)` (Day 5). **D5 — treating `b[0]` as bytes.** Indexing `bytes` gives an `int`, so `data[0] == b"a"` is always `False`. Compare with `ord("a")`, or slice with `data[0:1]` (Day 5). **D6 — decoding piece by piece.** `"".join(t.decode() for t in tokens)` breaks characters whose bytes are split across two tokens. Join all the bytes first, then decode once (Day 5). **D7 — the wrong `errors=` mode.** The spec says invalid bytes become `⍰`, but the code uses the default strict mode and crashes, or the other way round. **D8 — `str()` on bytes.** `str(b"hi")` is the six-character string `"b'hi'"`, not `"hi"`. Converting bytes to text needs `.decode()`. **D9 — parsing in the wrong base.** `int("ff")` raises an error. Hex digits need `int("ff", 16)`. Similarly, binary needs `int(s, 2)`. **D10 — missing zero padding.** `f"{10:x}"` gives `"a"`, but a two-digit hex format needs `"0a"`, i.e. `f"{10:02x}"`. Without padding, the output has the wrong length and can't be parsed back. **D11 — the wrong byte order.** `to_bytes(4, "little")` where the format says big-endian (or the reverse) gives wrong numbers without any error (Day 5, section 5). **D12 — sorting digit strings as text.** `sorted(["10", "9"])` gives `["10", "9"]`, because strings compare character by character. Numeric order needs `key=int` (Day 2, section 3).

E. Algorithm logic and state

Where they live: in the *logic* of the algorithm: when it stops, what it remembers between steps, and the order in which it does things. **What they look like:** a test that hangs (infinite loop or endless recursion); results that are right after the first step and wrong afterwards; one step too many or too few. **How to check:** trace two or three iterations by hand, writing down the full state (variables, list and dict contents) after each one, and compare with what the code does. **E1 — a missing or wrong base case.** A recursive function needs a case that *doesn't* call itself. If it's missing, or doesn't cover every input that can reach it, the recursion never ends:

```
def power(base, exp):
    """base ** exp for exp >= 0."""
    if exp == 1:                # BUG: power(2, 0) recurses forever (exp goes 0, -1, -2, ...)
        return base
    return base * power(base, exp - 1)
```

E2 — an infinite `while` loop. Some path through the loop body forgets to advance the variable the condition depends on (Day 1, section 4). **E3 — state that isn't reset (or is reset too often).** A counter or dict that should start empty for each item (each line, each word) is created once before the loop, so counts from earlier items leak into later ones. The reverse also happens: an accumulator that should survive across items is reset inside the loop, so only the last item counts. **E4 — updates in the wrong order.** Here the second line uses the *new* value of `a`, not the old one:

```
a = b
b = a + b               # BUG: this is b + b, because a was just overwritten
a, b = b, a + b         # correct: the right-hand side is computed first, from the old values
```

E5 — the wrong stopping condition. "Stop when no pair occurs more than once" means stop when the best count is below 2. Writing `< 1` instead lets the loop continue one step too far. Read the stop rule in the spec word by word. **E6 — leaving too early.** A `return` where the code should `break` or `continue`, or an `else` attached to the wrong `if`, makes the function finish before it has looked at everything. **E7 — checks in an order that makes some unreachable.** With `if b >= 0xC0: ... elif b >= 0xE0: ...`, the second branch can never run, because anything that's `>= 0xE0` is also `>= 0xC0` and is caught by the first (Day 5). **E8 — choosing by the wrong priority.** When several options are available, the spec says which to pick. The tokenizer must apply the merge learned *earliest* (lowest id). Code that uses `max` instead of `min` picks the latest one, and still produces plausible-looking output.

How to *find* them fast

The catalog tells you *what* to look for. Here's *where* to look, in order of how often it pays off:

1. **Diff the spec against the code line by line.** For each spec sentence, find the code that implements it. Planted bugs often sit exactly where a spec detail ("inclusive", "ties", "1-based", "bytes not characters", "ignore case") should have been implemented.
2. **Look hard at constants and operators:** every `0`, `1`, `-1`, `+1`, `<`, `<=`, `256`, `0x80` etc.
3. **Check every loop's bounds and its post-loop handling.**
4. **Check every return statement:** right variable? right indentation? copy vs alias?
5. **Hand-run the smallest failing test.** Two or three iterations by hand usually reveal it.

Steps 1–4 are about *reading*, and you can do them before running anything at all, during the 10 minutes of reading at the start of the exam. Step 5 is what you do when reading doesn't reveal the bug. One more trick that works surprisingly often. Since a planted bug is usually "one token changed", look at each suspicious token and ask: *what would the alternative be, and which version matches the spec?* `<` or `<=`? `i` or `j`? `max` or `min`? `0` or `1`? `a` or `b`? Usually only one of the two is plausible, and you can check it in seconds.

Background for the exercises

Two of today's exercises are built on classic algorithms you may not have met. Here's what you need to know about each before you start.

Binary search (ex02)

The problem. You have a **sorted** list and want to find a value in it, or the position where it would go. The obvious way is to look at every item in turn. For a million items, that's up to a million comparisons. **Binary search** needs about 20. **The idea.** It's how you'd look up a word in a paper dictionary. Open it in the middle. If your word comes earlier in the alphabet, you can forget the whole second half. Open the first half in the middle, and so on. Each comparison throws away half of what's left: a million, then 500,000, then 250,000… down to one in about 20 steps. **In code.** We keep two positions, `lo` and `hi`, marking the part of the list where the answer can still be. Each step looks at the middle position `mid` and moves `lo` or `hi` to throw away the half that can't contain the answer. The version below finds the **first position whose value is `>= x`**, which is also where `x` would be inserted to keep the list sorted:

```
def lower_bound(a, x):
    """First index i with a[i] >= x (len(a) if none)."""
    lo, hi = 0, len(a)            # the answer is somewhere in lo..hi; hi = len(a) means
        "after everything"
    while lo < hi:                # while there's more than one candidate
        mid = (lo + hi) // 2      # middle position (// so it's an int!)
        if a[mid] < x:
            lo = mid + 1          # a[mid] is too small, so the answer is after mid
        else:
            hi = mid              # a[mid] >= x: mid might be the answer itself, so keep it
    return lo
```

Let's trace `lower_bound([1, 3, 5, 7], 5)`:

lo	hi	mid	a[mid]	decision
0	4	2	5	5 is not < 5, so the answer is at 2 or before: hi = 2

lo	hi	mid	a[mid]	decision
0	2	1	3	3 < 5, so the answer is after 1: lo = 2
2	2	—	—	lo == hi, stop: answer 2

The two branches are deliberately different. When `a[mid]` is too small, `mid` itself can't be the answer, so `lo` jumps *past* it (`mid + 1`). When `a[mid]` is big enough, `mid` *might* be the answer, so `hi` stops *at* it (`mid`), keeping it in range. Every classic binary-search bug breaks one of these details:

- `lo = mid` instead of `lo = mid + 1`: when `hi == lo + 1`, `mid` equals `lo`, nothing changes, and the loop runs forever.
- `hi = len(a) - 1` at the start: the answer "after everything" can never be returned.
- `a[mid] <= x` instead of `<`: you get the first position with a value *greater than* `x` (that's called `upper_bound`).
- `/` instead of `//`: `mid` is a float and indexing crashes.

Today's file also contains three relatives. **`count_occurrences`** is `upper_bound - lower_bound`: the number of positions holding exactly `x`. **Searching a rotated sorted list** like `[4, 5, 6, 1, 2, 3]` (a sorted list that has been "rotated" so it starts in the middle) relies on one fact: when you split it at `mid`, at least one of the two halves is properly sorted, and you can check whether `x` falls in that half's range. The **integer square root** `isqrt(n)` uses binary search on *possible answers* rather than on a list: find the largest `r` with `r * r <= n`.

A calculator: tokenizer + shunting-yard (ex03)

The problem. Compute the value of a string like `"2 + 3 * (4 - 1)"`. The difficulty is that you can't just go left to right: multiplication must happen before addition, and parentheses override everything. Programs that do this (calculators, spreadsheet formulas, programming-language interpreters) usually work in three stages. **Stage 1: tokenize.** First, cut the string into meaningful pieces, called **tokens**. Spaces are skipped, a run of digits becomes one number, and each operator or parenthesis becomes its own token:

```
"2 + 3*(4-1)"   ->   [2, "+", 3, "*", "(", 4, "-", 1, ")"]
```

This is a scanning loop from Day 2. One subtle part is the minus sign. In `5 - 2` it means *subtract*, but in `-2` or `3 * -2` it means *negative*. The rule is: a `-` at the very start, or right after another operator or `(`, is a negation. The tokenizer marks those as a different token, `"u-"` ("unary minus", unary meaning it takes just one number). **Stage 2: reorder into Reverse Polish Notation. Reverse Polish Notation (RPN)** is a way of writing expressions where each operator comes *after* the numbers it applies to:

Normal notation	RPN
`2 + 3`	`2 3 +`
`2 + 3 * 4`	`2 3 4 * +`
`(2 + 3) * 4`	`2 3 + 4 *`

RPN never needs parentheses or precedence rules: the order of the tokens already says what to compute first. The **shunting-yard algorithm** converts normal notation to RPN using two lists: an **output** and a **stack of waiting operators**. (A stack is a list where you only add and remove at the end, Day 1.) It goes through the tokens one by one:

- A **number** goes straight to the output.
- An **operator** first moves to the output every waiting operator that must be computed *before* it (one with higher precedence, or equal precedence for left-to-right operators like `-`), and then waits on the stack itself.

- A `(` goes onto the stack. A `)` moves operators to the output until it reaches the matching `(`, which is thrown away.
- At the end, all remaining waiting operators move to the output.

Let's trace `2 + 3 * 4`:

Token	Action	Output	Stack
2	number → output	2	
+	stack is empty, so wait	2	+
3	number → output	2 3	+
*	+ has lower precedence than *, so it stays; * waits	2 3	+ *
4	number → output	2 3 4	+ *
end	move all waiting operators to the output, last first	2 3 4 * +	

Associativity decides what happens when two operators have *equal* precedence. Subtraction goes left to right: `8 - 3 - 2` means `(8 - 3) - 2 = 3`, so an equal operator already waiting is moved out first. Exponentiation (`^`) goes right to left: `2 ^ 3 ^ 2` means `2 ^ (3 ^ 2) = 512`, so an equal `^` waiting on the stack is *not* moved out. In the code this is the difference between a `>` and a `>=` in one comparison, which makes it a very likely bug location. **Stage 3: evaluate the RPN.** Use a stack of values. A number is pushed. An operator pops the top two values, applies itself, and pushes the result. Careful: the value popped *first* is the *right-hand* operand.

```
evaluating  2 3 4 * +
  push 2              -> [2]
  push 3              -> [2, 3]
  push 4              -> [2, 3, 4]
  "*": pop 4, pop 3   -> push 3 * 4 = 12  -> [2, 12]
  "+": pop 12, pop 2  -> push 2 + 12 = 14 -> [14]
result: 14
```

At the end exactly one value must remain. Anything else means the expression was malformed. Where bugs hide in calculator code:

- **The operand order when popping.** `a - b` and `b - a` only differ for `-`, `/` and `^`, so tests using `+` and `*` still pass, which makes this bug easy to miss.
- **The precedence comparison** (`>` vs `>=`): that's associativity.
- **Detecting unary minus**, i.e. checking what came before the `-`.
- **Number conversion**: `"2.5"` must become a float, not be truncated to an int.
- **Error checks**: unbalanced parentheses, or values left over at the end.

The toolbox: Python you'll meet in today's exercises

Most of what the exercises use was explained on Days 1–6. These are the few remaining techniques, plus reminders of ones that are easy to misread.

Using a module: `import copy`

Some tools aren't built in but live in **modules** from Python's standard library, which is always available, including in the exam. `import copy` at the top of a file makes the module available, and you use its functions with a dot: `copy.deepcopy(cfg)` (Day 3, section 7). You'll see the same pattern with `heapq`, `collections` and `unittest`.

Replacing a list's contents in place: `nums[:] = ...`

`nums = [x for x in nums if x >= 0]` builds a new list and points the **local name** `nums` at it. The caller's list is untouched. If the spec says "modify the list in place", assign to a **slice** instead:

```
nums[:] = [x for x in nums if x >= 0]
```

`nums[:]` on the left of = means "all the positions of *this* list": the existing list object keeps its identity, and its contents are replaced. Everyone holding a reference to it sees the change.

Swapping two positions

Unpacking (Day 1, section 8) swaps two items without a temporary variable, because the right side is evaluated before anything is assigned:

```
out[i], out[i + 1] = out[i + 1], out[i]
```

Merging two sorted lists with two pointers

`merge_sorted` walks through both lists at once, with an index into each (`i` and `j`). At each step it compares the two current items, appends the smaller one, and moves that list's index forward. When one list runs out, the loop stops, and **whatever is left of the other list** (already sorted, and all at least as big) is added with `extend`:

```
a = [1, 4, 9], b = [2, 3]
1 < 2 -> take 1 (i=1)     4 > 2 -> take 2 (j=1)     4 > 3 -> take 3 (j=2, b is used up)
leftovers: a[1:] = [4, 9]  ->  [1, 2, 3, 4, 9]
```

Forgetting the leftovers is catalog bug A5.

The median

The median is the middle value of the **sorted** data. With an odd number of items there's exactly one middle, `s[n // 2]`: for 5 items that's position 2. With an even number there are two middles, `s[n // 2 - 1]` and `s[n // 2]`, and the median is their average: for `[1, 2, 3, 4]` it's `(2 + 3) / 2 = 2.5`. `if n % 2:` is a short way of writing "if `n` is odd", because `n % 2` is 1 (truthy) for odd numbers and 0 (falsy) for even ones.

A dict as a lookup table

The calculator keeps operator precedences in a dict, `PREC = {"+": 1, "-": 1, "*": 2, ...}`. That makes two questions one-liners:

- "How strong is this operator?" is `PREC[t]`.
- "Is this token an operator?" is `t in PREC`, because `in` on a dict checks its keys (Day 3).

`c in "+-*/^()"` asks whether the one-character string `c` occurs in that string, which is a compact way of checking "is `c` one of these characters?".

A list that mixes numbers and strings

The token list holds both numbers (`2`, `3.5`) and strings (`"+"`, `"("`). To tell them apart, the code asks `isinstance(t, (int, float))` (Day 3, section 9): numbers go one way, operators the other. Note that the number `2` and the string `"2"` are different tokens. Converting the text of a number at the right moment, with `int(...)` or `float(...)`, is part of the tokenizer's job. `prev = tokens[-1] if tokens else None` (Day 1, section 8) reads "the previous token, or `None` if there isn't one yet", which is how the tokenizer knows whether a `-` is the very first token.

Pop, change, push back

`st.append(-st.pop())` reads from the inside out: take the top value off the stack, negate it, and push the result back. That's how unary minus is applied in one line.

Exercises

Practice lab

Target: under 2 minutes per bug. Write the catalog code (for example `A4`) in a comment next to each fix.

File	Type	What
`exercises/ex01_bug_zoo.py`	**debug**	22 tiny functions, one catalog bug each. Target: < 2 min per bug
`exercises/ex02_binary_search.py`	**debug**	binary-search family, 5 bugs
`exercises/ex03_calculator.py`	**debug**	tokenizer + shunting-yard expression evaluator, 7 bugs. 40 min

```
python run.py 7
```

How to approach each:

- **ex01** is a speed drill. Each function is tiny and its bug is a single catalog entry, so the goal is to *recognise* it, not to analyse it. If you need more than two minutes for one, read its docstring again and run through the catalog categories.
- **ex02** and **ex03** are closer to the exam: several bugs interacting inside one algorithm, with the spec at the top of the file. Read the spec *and* the background above before running any tests.

For each bug you fix, write the catalog code (e.g. `A4`) in a comment next to the fix. That's how you build the reflex.

End-of-day checklist

Tick every box

- [ ] I can name the catalog code (A4, B3, C10, ...) for each bug I fixed.
- [ ] I ran `python run.py 7` and the full suite passes.
- [ ] I added tonight's bugs to `bug_journal.md`.

Exam lens

- Planted bugs are one-token changes: `<` versus `<=`, `i` versus `i+1`, `+` versus `-`, a mask, `is` versus `==`.
- Check loop bounds, post-loop handling, and the indentation of every `return`.
- Name the catalog code in your journal; recognition is the skill being tested.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.